DATA STRUCTURES

LAB VIVA QUESTION BANK

English Answers + Malayalam Explanations

Prepared around the 8 programs selected for your lab exam

01 Insertion Sort	02 Quick Sort
03 Merge Sort	04 Infix to Postfix
05 Circular Queue	06 Singly Linked List
07 Doubly Linked List	08 BST Dictionary

Includes the numbered questions from the uploaded 93-question viva bank, grouped by topic.

Short, viva-friendly answers • Simple Malayalam meanings • Important clarification notes

How to use this booklet

Each question has a short English answer followed by a Malayalam explanation. The original question-bank number is kept as Q1–Q93. Questions marked P1, P2, etc. are extra practical questions based on the programs you uploaded; they are not numbered items from the original viva bank.

Important source notes: A few questions in the original file contain unclear or imprecise wording. These are flagged in the relevant answers (for example Q35, Q40, Q82, Q88 and the Linked List classification questions). The uploaded bank skips Q6 and Q7.

Contents

PART 1 - Questions related to the 8 lab programs

- Sorting concepts and the Insertion / Quick / Merge Sort programs
- Infix to Postfix - Stack and expression conversion
- Circular Queue - Queue operations
- Singly Linked List - Nodes, pointers and memory
- Doubly Linked List - Previous and next pointers
- BST Dictionary - Binary Search Tree

PART 2 - Other general viva questions

- General Data Structure fundamentals
- Other sorting and searching questions
- Other trees, graphs and structures
- Other recursion and function questions

PART 1 | QUESTIONS RELATED TO THE 8 LAB PROGRAMS

These questions are grouped under the programs named by your teacher. Some shared sorting and linked-list concepts are placed where they are most useful for revision.

COMMON SORTING CONCEPTS | Insertion Sort, Quick Sort and Merge Sort

Q14 What is an algorithm?

Answer: An algorithm is a finite, step-by-step set of instructions used to solve a problem or perform a task. The steps must be arranged in a clear order and should produce the required result.

മലയാളം: ഒരു പ്രശ്നം പരിഹരിക്കാനോ ഒരു ജോലി ചെയ്യാനോ ക്രമമായി പാലിക്കുന്ന ഘട്ടങ്ങളെയാണ് Algorithm എന്ന് പറയുന്നത്. ഓരോ ഘട്ടവും വ്യക്തവും ക്രമത്തിലുള്ളതുമായിരിക്കണം.

Q15 Why do we need algorithm analysis?

Answer: Algorithm analysis estimates the resources needed by an algorithm, mainly execution time and memory space. It helps compare different solutions and choose an efficient one.

മലയാളം: ഒരു Algorithm പ്രവർത്തിക്കാൻ എത്ര സമയം, എത്ര memory എന്നിവ ആവശ്യമാണ് എന്ന് കണക്കാക്കുന്നതാണ് Algorithm Analysis. പല രീതികളിൽ ഏതാണ് കൂടുതൽ കാര്യക്ഷമമെന്ന് കണ്ടെത്താൻ ഇത് സഹായിക്കുന്നു.

Q27 Which sorting algorithm is considered the fastest? Why?

Answer: There is no single sorting algorithm that is fastest for every input. Quick Sort is often very fast in practice because it partitions arrays efficiently and usually runs in O(n log n) time. Its worst-case time is O(n^2).

മലയാളം: എല്ലാ സാഹചര്യങ്ങളിലും ഏറ്റവും വേഗമുള്ള ഒരേയൊരു sorting algorithm ഇല്ല. സാധാരണ സാഹചര്യങ്ങളിൽ Quick Sort വേഗത്തിലാണ് പ്രവർത്തിക്കുന്നത്. അതിന്റെ ശരാശരി സമയം O(n log n) ആണ്; worst case O(n^2) ആകാം.

Q30 What is asymptotic analysis of an algorithm?

Answer: Asymptotic analysis studies how an algorithm's time or space requirement grows as the input size n becomes large. It helps compare algorithms independently of a particular computer.

മലയാളം: Input എണ്ണം n കൂടുമ്പോൾ Algorithm-ന്റെ സമയം അല്ലെങ്കിൽ memory എങ്ങനെ കൂടുന്നു എന്ന് പഠിക്കുന്ന രീതിയാണ് Asymptotic Analysis. കമ്പ്യൂട്ടറിന്റെ വേഗതയെ ആശ്രയിക്കാതെ algorithms താരതമ്യം ചെയ്യാൻ ഇത് സഹായിക്കുന്നു.

Q31 What are asymptotic notations?

Answer: The main asymptotic notations are Big O, Big Omega and Big Theta. Big O gives an upper bound, Big Omega gives a lower bound, and Big Theta gives a tight bound on growth.

മലയാളം: Algorithm-ന്റെ വളർച്ച വിവരിക്കാൻ ഉപയോഗിക്കുന്ന notation-കളാണ് Big O, Big Omega, Big Theta. Big O upper bound, Omega lower bound, Theta tight bound സൂചിപ്പിക്കുന്നു.

Q72 What factors affect the efficiency of sorting a file?

Answer: Important factors include the time needed to write the program, the machine time required to run it, and the memory space required by the program.

മലയാളം: Sorting program എഴുതാൻ വേണ്ട സമയം, അത് run ചെയ്യാൻ വേണ്ട computer time, program-ന് ആവശ്യമായ memory space എന്നിവയാണ് കാര്യക്ഷമതയെ ബാധിക്കുന്ന ഘടകങ്ങൾ.

Q83 Sorting is not possible by using which method: insertion, selection, exchange or deletion?

Answer: Deletion alone is not a sorting method. Insertion, selection and exchange can each form the basis of sorting algorithms, but simply deleting elements does not arrange the remaining data in sorted order.

മലയാളം: Deletion മാത്രം ഉപയോഗിച്ച് data sort ചെയ്യാൻ കഴിയില്ല. Insertion, Selection, Exchange എന്നിവയെ അടിസ്ഥാനമാക്കി sorting algorithms ഉണ്ടാക്കാം. എന്നാൽ delete ചെയ്യുന്നത് മാത്രം data-യെ ക്രമത്തിൽ അടുക്കില്ല.

Insertion Sort - program understanding

P1 What is Insertion Sort?

Answer: Insertion Sort builds a sorted portion of the array one element at a time. It takes the next element as the key, shifts larger elements one position to the right, and inserts the key in its correct position.

മലയാളം: Insertion Sort-ൽ array-യിലെ elements ഓരോന്നായി sorted ഭാഗത്തേക്ക് ചേർക്കുന്നു. അടുത്ത element key ആക്കുന്നു; അതിനേക്കാൾ വലിയ elements വലത്തേക്ക് shift ചെയ്ത് key ശരിയായ സ്ഥാനത്ത് ഇടുന്നു.

Note: *This is an additional practical question based on your program, not a numbered question in the uploaded viva bank.*

P2 What is the key in Insertion Sort?

Answer: The key is the current element selected to be inserted into the already-sorted part of the array.

മലയാളം: ഇതിനകം sort ചെയ്ത ഭാഗത്തിലേക്ക് ശരിയായ സ്ഥാനത്ത് ചേർക്കാൻ ഇപ്പോൾ തിരഞ്ഞെടുക്കുന്ന element ആണ് key.

Note: *Additional practical question.*

P3 What is the time complexity of Insertion Sort?

Answer: Best case: O(n), when the array is already sorted. Average and worst cases: O(n^2). Its extra space complexity is O(1).

മലയാളം: Array നേരത്തെ sorted ആണെങ്കിൽ best case O(n) ആണ്. Average, worst case O(n^2) ആണ്. അധിക memory complexity O(1) ആണ്.

Note: *Additional practical question.*

Quick Sort - program understanding

Q32 What are some examples of divide-and-conquer algorithms?

Answer: Quick Sort and Merge Sort are examples. Quick Sort partitions the array around a pivot and sorts the partitions recursively. Merge Sort divides the array into halves, sorts them recursively, and merges the sorted halves.

മലയാളം: ഒരു വലിയ പ്രശ്നത്തെ ചെറിയ ഭാഗങ്ങളാക്കി പരിഹരിക്കുന്ന രീതിയാണ് Divide and Conquer. Quick Sort-ൽ pivot ഉപയോഗിച്ച് വിഭജിക്കുന്നു. Merge Sort-ൽ array halves ആയി വിഭജിച്ച് sort ചെയ്ത ശേഷം merge ചെയ്യുന്നു.

P4 What is a pivot in Quick Sort?

Answer: A pivot is the element used to partition the array. Elements smaller than the pivot are placed on one side and larger elements on the other side, according to the partition scheme.

മലയാളം: Quick Sort-ൽ array വിഭജിക്കാൻ തിരഞ്ഞെടുക്കുന്ന element ആണ് pivot. Pivot-നെക്കാൾ ചെറിയ values ഒരു വശത്തും വലിയ values മറുവശത്തും ക്രമീകരിക്കുന്നു.

Note: *Additional practical question.*

P5 What is the time complexity of Quick Sort?

Answer: Average and best cases are O(n log n). The worst case is O(n^2), for example when partitions are repeatedly very unbalanced. Typical implementations use O(log n) average recursion stack space.

മലയാളം: Quick Sort-ന്റെ best, average time O(n log n) ആണ്. വളരെ അസമമായ partition തുടർച്ചയായി വന്നാൽ worst case O(n^2) ആകും.

Note: *Additional practical question.*

Merge Sort - program understanding

Q28 What is Merge Sort? How does it work?

Answer: Merge Sort is a divide-and-conquer sorting algorithm. It divides the array into smaller parts until each part has one element, then merges the parts in sorted order until one sorted array is produced.

മലയാളം: Merge Sort ഒരു Divide and Conquer algorithm ആണ്. Array ചെറിയ ഭാഗങ്ങളായി വിഭജിച്ച് ഓരോ ഭാഗത്തും ഒരു element മാത്രം വരുന്നതുവരെ പോകുന്നു. ശേഷം sorted order-ൽ ഭാഗങ്ങൾ merge ചെയ്ത് പൂർണ്ണമായും sort ചെയ്യുന്നു.

P6 What is the time complexity of Merge Sort?

Answer: Merge Sort takes O(n log n) time in the best, average and worst cases. A typical array implementation uses O(n) extra memory for merging.

മലയാളം: Merge Sort-ന്റെ best, average, worst time complexity എല്ലാം O(n log n) ആണ്. Merge ചെയ്യാൻ സാധാരണ array implementation-ൽ O(n) അധിക memory ആവശ്യമാണ്.

Note: *Additional practical question.*

P7 Why is a temporary array used in Merge Sort?

Answer: A temporary array stores elements while two sorted halves are merged. It helps place elements in the correct sorted order before copying them back to the original array.

മലയാളം: രണ്ട് sorted ഭാഗങ്ങൾ merge ചെയ്യുമ്പോൾ elements ക്രമത്തിൽ താൽക്കാലികമായി സൂക്ഷിക്കാൻ temporary array ഉപയോഗിക്കുന്നു. ശേഷം അവ original array-ലേക്ക് copy ചെയ്യുന്നു.

Note: *Additional practical question.*

INFIX TO POSTFIX | Stack and Expression Conversion

Q16 What is a Stack?

Answer: A stack is a linear data structure in which insertion and deletion take place at one end called TOP. It follows LIFO: Last In, First Out.

മലയാളം: Stack ഒരു linear data structure ആണ്. ഇതിൽ insertion-ഉം deletion-ഉം TOP എന്ന ഒരേയൊരു അറ്റത്താണ് നടക്കുന്നത്. അവസാനം ചേർത്തത് ആദ്യം പുറത്തുവരും (LIFO).

Q17 Where are Stacks used?

Answer: Stacks are used in expression conversion and evaluation, syntax parsing, parenthesis matching, string reversal, backtracking and function-call management.

മലയാളം: Expression conversion/evaluation, syntax checking, parentheses matching, string reverse, backtracking, function calls എന്നിവയിൽ Stack ഉപയോഗിക്കുന്നു.

Q18 What operations can be performed on a Stack?

Answer: The basic operations are PUSH (insert an item), POP (remove the top item) and PEEK (read the top item without removing it). Overflow happens when pushing into a full stack; underflow happens when popping from an empty stack.

മലയാളം: Stack-ലെ പ്രധാന operations PUSH (ചേർക്കുക), POP (മുകളിലെ element നീക്കുക), PEEK (നീക്കാതെ മുകളിലെ element കാണുക) എന്നിവയാണ്. Full stack-ലേക്ക് push ചെയ്താൽ overflow; empty stack-ൽ pop ചെയ്താൽ underflow.

Q19 What is a Postfix expression?

Answer: A postfix expression places the operator after its operands. For example, the infix expression A + B is written as A B + in postfix notation.

മലയാളം: Postfix expression-ൽ operator operands-ന്റെ ശേഷമാണ് വരുന്നത്. ഉദാഹരണം: Infix A+B ആണെങ്കിൽ Postfix AB+ ആണ്.

Q25 Where can Stack data structures be used?

Answer: Stacks are used in expression evaluation, backtracking, memory management and function calling and return. These are also examples listed in the viva bank.

മലയാളം: Expression evaluation, backtracking, memory management, function call-ഉം return-ഉം എന്നിവയിൽ Stack ഉപയോഗിക്കുന്നു.

Q26 What is the difference between PUSH and POP?

Answer: PUSH adds an element to the top of a stack. POP removes the top element. PUSH on a full stack causes overflow; POP on an empty stack causes underflow.

മലയാളം: PUSH ഒരു element Stack-ന്റെ മുകളിൽ ചേർക്കുന്നു. POP മുകളിലുള്ള element നീക്കുന്നു. Full stack-ൽ PUSH ചെയ്താൽ overflow; empty stack-ൽ POP ചെയ്താൽ underflow.

Q75 Why are parentheses not required in Postfix or Prefix expressions?

Answer: The position of each operator determines the order of operations in Prefix and Postfix notation, so parentheses are not needed to show precedence or grouping.

മലയാളം: Prefix, Postfix notation-കളിൽ operator-ന്റെ സ്ഥാനം തന്നെ operation-ന്റെ ക്രമം വ്യക്തമാക്കുന്നു. അതിനാൽ precedence/grouping കാണിക്കാൻ parentheses ആവശ്യമില്ല.

Q81 What notations are used for arithmetic expressions in Prefix and Postfix forms?

Answer: Prefix notation is also called Polish notation. Postfix notation is also called Reverse Polish notation.

മലയാളം: Prefix notation-നെ Polish notation എന്നും Postfix notation-നെ Reverse Polish notation എന്നും വിളിക്കുന്നു.

Q82 Convert the expression shown in the question bank into Prefix and Postfix notation.

Answer: The uploaded question bank appears to have lost some operator symbols in the printed infix expression. It shows the answer as Prefix: ^ * + A B C D E + F G; Postfix: A B + C * D E F G + ^. Because the infix expression is incomplete/ambiguous, confirm the missing operators with your teacher before memorising this conversion.

മലയാളം: നിങ്ങൾ നൽകിയ question bank-ൽ infix expression-ലെ ചില operator ചിഹ്നങ്ങൾ നഷ്ടപ്പെട്ടതായി തോന്നുന്നു. അതിനാൽ നൽകിയ Prefix/Postfix answer മാത്രം നോക്കി conversion പഠിക്കരുത്. ശരിയായ expression teacher-നോട് ഉറപ്പാക്കിയശേഷം conversion practice ചെയ്യുക.

Note: *Source question is incomplete/ambiguous; answer reproduced cautiously from the uploaded bank, not treated as verified.*

P8 What is operator precedence?

Answer: Operator precedence decides which operator is evaluated first. For example, multiplication has higher precedence than addition in ordinary arithmetic expressions.

മലയാളം: ഒരു expression-ൽ ഏത് operator ആദ്യം process ചെയ്യണം എന്ന് തീരുമാനിക്കുന്ന നിയമമാണ് operator precedence. സാധാരണ arithmetic-ൽ multiplication-ന് addition-നെക്കാൾ ഉയർന്ന precedence ഉണ്ട്.

Note: *Additional practical question.*

P9 What is associativity?

Answer: Associativity decides the direction of evaluation when operators have the same precedence. For example, subtraction is generally left-associative.

മലയാളം: ഒരേ precedence ഉള്ള operators ഒന്നിലധികം വന്നാൽ ഏത് ദിശയിൽ evaluate ചെയ്യണം എന്ന് associativity തീരുമാനിക്കുന്നു. Subtraction സാധാരണ left-to-right ആണ്.

Note: *Additional practical question.*

CIRCULAR QUEUE | Queue Operations

Q20 What is a Queue data structure?

Answer: A queue is a linear data structure that follows FIFO (First In, First Out). Insertion takes place at the REAR and deletion takes place at the FRONT.

മലയാളം: Queue ഒരു linear data structure ആണ്. ആദ്യം ചേർത്ത element ആദ്യം പുറത്തുവരും (FIFO). REAR-ൽ insertion, FRONT-ൽ deletion നടക്കുന്നു.

Q21 List some applications of Queue data structures.

Answer: Queues are used for printer and CPU job scheduling, call-centre waiting lists, asynchronous data transfer and buffering in media players or communication systems.

മലയാളം: Printer jobs, CPU scheduling, call-centre waiting list, data transfer, media player buffering തുടങ്ങിയവയിൽ Queue ഉപയോഗിക്കുന്നു.

Q22 What is a Deque?

Answer: A Deque (double-ended queue) is a queue in which insertion and deletion can be performed at both the FRONT and REAR ends.

മലയാളം: Deque അഥവാ Double-ended Queue-ൽ FRONT-ലും REAR-ലും insertion-ഉം deletion-ഉം ചെയ്യാൻ കഴിയും.

Q23 What operations can be performed on Queues?

Answer: Common operations include enqueue (insert at rear), dequeue (delete from front), initialize, isEmpty, reading the front element and reading the rear element.

മലയാളം: Enqueue (rear-ൽ ചേർക്കുക), Dequeue (front-ൽ നിന്ന് നീക്കുക), initialize, isEmpty, front element കാണുക, rear element കാണുക എന്നിവയാണ് Queue operations.

Q60 What are Overflow and Underflow?

Answer: Overflow occurs when an insertion is attempted but the data structure has no available capacity. Underflow occurs when deletion is attempted on an empty data structure.

മലയാളം: ചേർക്കാൻ ഇടമില്ലാത്തപ്പോൾ insert ചെയ്യാൻ ശ്രമിക്കുന്നത് Overflow ആണ്. Empty ആയ data structure-ൽ നിന്ന് delete ചെയ്യാൻ ശ്രമിക്കുന്നത് Underflow ആണ്.

Q62 What is a Queue?

Answer: A queue is an ordered collection in which elements are inserted at the REAR and deleted from the FRONT. It follows the FIFO rule.

മലയാളം: Queue-ൽ elements REAR-ൽ ചേർക്കുകയും FRONT-ൽ നിന്ന് നീക്കുകയും ചെയ്യുന്നു. ഇത് FIFO rule പിന്തുടരുന്നു.

Q63 What is a Priority Queue?

Answer: A priority queue removes elements according to their priority rather than simply their arrival order. The source describes ascending priority queues (smallest item removed first) and descending priority queues (largest item removed first).

മലയാളം: Priority Queue-ൽ element വന്ന ക്രമം മാത്രം നോക്കാതെ അതിന്റെ priority അനുസരിച്ചാണ് നീക്കം ചെയ്യുന്നത്. Ascending-ൽ ചെറിയ value-ക്കും descending-ൽ വലിയ value-ക്കും മുൻഗണന ലഭിക്കും.

Q79 What is the minimum number of queues needed to implement a Priority Queue?

Answer: The uploaded question bank gives the answer as two: one queue stores the data and another stores priorities. This is the expected answer from that bank; implementations can vary.

മലയാളം: നിങ്ങളുടെ question bank-ൽ ഉത്തരം രണ്ട് Queue എന്നാണ്: ഒന്ന് data സൂക്ഷിക്കാൻ, മറ്റൊന്ന് priority സൂക്ഷിക്കാൻ. ഇത് ആ question bank-ൽ നൽകിയിരിക്കുന്ന ഉത്തരമാണ്; implementation അനുസരിച്ച് വ്യത്യാസം വരാം.

P10 What is a Circular Queue?

Answer: A circular queue is a queue in which the last array position logically connects back to the first position. It reuses free positions created by deletions.

മലയാളം: Circular Queue-ൽ array-യുടെ അവസാന position വീണ്ടും ആദ്യ position-ലേക്ക് ചുറ്റി വരുന്നു. Delete ചെയ്തതിലൂടെ ഒഴിവായ സ്ഥാനങ്ങൾ വീണ്ടും ഉപയോഗിക്കാം.

Note: *Additional practical question based on your program.*

P11 What is the purpose of (rear + 1) % MAX?

Answer: It moves the rear index to the next position and wraps it to index 0 after the last array index is reached.

മലയാളം: Rear അടുത്ത position-ലേക്ക് നീങ്ങുന്നു. അവസാന index കഴിഞ്ഞാൽ modulo ഉപയോഗിച്ച് വീണ്ടും 0-ലേക്ക് മടങ്ങുന്നു.

Note: *Additional practical question.*

P12 What is the difference between Linear Queue and Circular Queue?

Answer: In a simple linear array queue, freed positions at the beginning may not be reused without shifting. A circular queue wraps the indices and reuses those positions.

മലയാളം: Linear Queue-ൽ തുടക്കത്തിൽ delete ചെയ്തതിനാൽ ഒഴിവായ സ്ഥലം ചില implementations-ൽ വീണ്ടും ഉപയോഗിക്കാൻ കഴിയില്ല. Circular Queue-ൽ index ചുറ്റി സഞ്ചരിക്കുന്നതിനാൽ അത് വീണ്ടും ഉപയോഗിക്കാം.

Note: *Additional practical question.*

SINGLY LINKED LIST | Nodes, Pointers and Memory

Q8 What is a Linked List data structure?

Answer: A linked list is a linear collection of nodes that are not required to occupy adjacent memory locations. Each node stores data and a pointer to the next node. The first node is referenced by head; the last node usually points to NULL.

മലയാളം: Linked List എന്നത് nodes ഉപയോഗിച്ച് data സൂക്ഷിക്കുന്ന linear structure ആണ്. Nodes തുടർച്ചയായ memory-ൽ വേണമെന്നില്ല. ഓരോ node-ലും data-യും അടുത്ത node-ന്റെ address-ഉം ഉണ്ടാകും. ആദ്യ node-നെ head സൂചിപ്പിക്കുന്നു; അവസാന node സാധാരണ NULL-ലേക്ക് link ചെയ്യും.

Q9 Are Linked Lists linear or non-linear data structures?

Answer: A linked list is normally classified as a linear data structure because its nodes are arranged in a sequence. The uploaded bank also mentions that classification can be described differently depending on the context of use.

മലയാളം: സാധാരണ Data Structures classification-ൽ Linked List ഒരു linear data structure ആണ്; nodes ഒരു ക്രമത്തിൽ ബന്ധിപ്പിച്ചിരിക്കുന്നു. നിങ്ങളുടെ question bank ഉപയോഗസാഹചര്യത്തെ ആശ്രയിച്ച് വ്യത്യസ്തമായി പറയാമെന്നും സൂചിപ്പിക്കുന്നു.

Note: *The source gives a context-dependent answer. For a standard viva definition, say that a linked list is linear.*

Q10 What are the advantages of a Linked List over an Array?

Answer: A linked list can grow or shrink dynamically. Insertions and deletions can be done by changing links without shifting many elements. It does not require a single contiguous memory block. However, it needs extra pointer memory and does not provide direct indexed access.

മലയാളം: Linked List-ന് ആവശ്യാനുസരണം വലുപ്പം കൂട്ടാനും കുറയ്ക്കാനും കഴിയും. Insert/delete ചെയ്യുമ്പോൾ array പോലെ പല elements shift ചെയ്യേണ്ടതില്ല. എന്നാൽ pointers-ന് അധിക memory വേണം; index ഉപയോഗിച്ച് നേരിട്ട് access ചെയ്യാൻ കഴിയില്ല.

Q13 What are Dynamic Data Structures? Give examples.

Answer: Dynamic data structures can grow or shrink while a program is running. Examples include linked lists, stacks, queues and dynamically allocated arrays.

മലയാളം: Program run ചെയ്യുമ്പോൾ ആവശ്യത്തിന് വലുപ്പം കൂടുകയോ കുറയുകയോ ചെയ്യുന്ന data structures ആണ് Dynamic Data Structures. Linked List, Stack, Queue എന്നിവ ഉദാഹരണങ്ങളാണ്.

Q40 Differentiate NULL and VOID.

Answer: In C, NULL is a macro/constant used to represent a null pointer value. void is a type used to indicate no value or a generic pointer type when written as void *. They are not interchangeable.

മലയാളം: C language-ൽ NULL എന്നത് ഒരു pointer ഒന്നിനെയും point ചെയ്യുന്നില്ലെന്ന് സൂചിപ്പിക്കുന്നു. void എന്നത് return value ഇല്ലാത്ത function-ലോ generic pointer (void *)-ലോ ഉപയോഗിക്കുന്ന type ആണ്. രണ്ടും ഒരേ അർത്ഥമല്ല.

Note: *The source wording is unclear; this is the standard C-language distinction.*

Q41 Does dynamic memory allocation help in managing data? How?

Answer: Yes. Dynamic memory allocation obtains memory at run time. Separately allocated nodes can be connected to build structures that grow or shrink as needed.

മലയാളം: Dynamic memory allocation run time-ൽ memory എടുക്കാൻ സഹായിക്കുന്നു. വേർതിരിച്ച് allocate ചെയ്ത nodes pointers ഉപയോഗിച്ച് ബന്ധിപ്പിച്ച് ആവശ്യത്തിന് structure വലുതാക്കാനോ ചെറുതാക്കാനോ കഴിയും.

Q42 How can you determine whether a Linked List has a loop?

Answer: Common methods include hashing or marking visited nodes, and Floyd's cycle-finding algorithm (slow and fast pointers).

മലയാളം: Linked List-ൽ loop ഉണ്ടോ എന്ന് കണ്ടെത്താൻ hashing, visited nodes mark ചെയ്യൽ, അല്ലെങ്കിൽ Floyd's cycle-finding algorithm (slow/fast pointers) ഉപയോഗിക്കാം.

Q55 What is a Linked List?

Answer: A linked list is a collection of nodes in which the order is maintained by pointers. Each node contains an information field and a link to the next node.

മലയാളം: Pointers ഉപയോഗിച്ച് ക്രമം നിലനിർത്തുന്ന nodes-ന്റെ കൂട്ടമാണ് Linked List. ഓരോ node-ലും data field-ഉം അടുത്ത node-ലേക്കുള്ള link-ഉം ഉണ്ടായിരിക്കും.

Q56 What are the advantages of Linked List over an Array (static data structure)?

Answer: Linked lists make insertion and deletion easier because links can be changed instead of shifting array elements. Nodes can occupy non-adjacent memory locations and memory can be allocated when required.

മലയാളം: Array-ൽ elements shift ചെയ്യുന്നതിന് പകരം links മാറ്റി insertion/deletion നടത്താം. Nodes തുടർച്ചയായ memory-ൽ ആവശ്യമില്ല; ആവശ്യമായപ്പോൾ memory allocate ചെയ്യാം.

Q57 Can Binary Search be applied efficiently to a sorted Linked List? Why?

Answer: It is generally not efficient. A linked list does not support direct indexed access to its middle element, so reaching the middle requires traversal. Binary Search is most useful when middle elements can be accessed directly.

മലയാളം: Sorted Linked List-ൽ Binary Search സാധാരണ കാര്യക്ഷമമല്ല. List-ൽ middle element-നെ index ഉപയോഗിച്ച് നേരിട്ട് കണ്ടെത്താനാവില്ല; അവിടെ എത്താൻ traversal വേണം.

Q58 What is a Free Pool?

Answer: A free pool is a list or collection of unused memory cells or nodes available for allocation.

മലയാളം: ഇനിയും ഉപയോഗിക്കാത്ത memory cells/nodes സൂക്ഷിച്ചിരിക്കുന്ന collection ആണ് Free Pool. പുതിയ node allocate ചെയ്യുമ്പോൾ ഇവയിൽ നിന്ന് memory ലഭിക്കും.

Q59 What is Garbage Collection?

Answer: Garbage collection is a process of reclaiming memory occupied by data that is no longer in use, so that the memory can be reused.

മലയാളം: ഇനി ആവശ്യമില്ലാത്ത data കൈവശം വെച്ചിരിക്കുന്ന memory തിരിച്ചുപിടിച്ച് വീണ്ടും ഉപയോഗിക്കാൻ ലഭ്യമാക്കുന്ന പ്രക്രിയയാണ് Garbage Collection.

Q61 What are the disadvantages of array implementations of Linked Lists?

Answer: The maximum number of nodes may need to be estimated in advance, and the allocated array space may remain reserved even when fewer nodes are used. This reduces flexibility compared with dynamically allocated nodes.

മലയാളം: Array ഉപയോഗിച്ച് Linked List implement ചെയ്യുമ്പോൾ nodes എത്ര വേണമെന്ന് മുൻകൂട്ടി കണക്കാക്കേണ്ടിവരും. ഉപയോഗിക്കാത്ത array space പോലും reserve ചെയ്തിരിക്കാം.

Q65 What are the disadvantages of representing a Stack or Queue using a Linked List?

Answer: Each linked-list node needs pointer storage in addition to data. Managing links and memory allocation also adds some overhead compared with a simple array implementation.

മലയാളം: Linked List ഉപയോഗിച്ച് Stack/Queue ഉണ്ടാക്കുമ്പോൾ data-ക്കൊപ്പം pointer-നും memory വേണം. Links-ഉം dynamic memory-ഉം manage ചെയ്യുന്നതിനും അധിക overhead ഉണ്ട്.

Q66 What is a Dangling Pointer and how can it be avoided?

Answer: A dangling pointer points to memory that has already been freed or is no longer valid. After freeing memory, set the pointer to NULL and do not dereference it.

മലയാളം: Free ചെയ്ത memory-യിലേക്ക് ഇപ്പോഴും point ചെയ്യുന്ന pointer ആണ് Dangling Pointer. Memory free ചെയ്തശേഷം pointer-നെ NULL ആക്കുകയും വീണ്ടും dereference ചെയ്യാതിരിക്കുകയും ചെയ്യണം.

Q67 What are the disadvantages of a linear list?

Answer: In a singly linked list, it is not possible to move directly to a previous node. To reach an earlier node, traversal must start again from the head. The external head pointer must also be preserved to access the list.

മലയാളം: Singly Linked List-ൽ പിന്നിലേക്ക് നേരിട്ട് പോകാൻ കഴിയില്ല. മുൻ node-ലേക്ക് എത്താൻ head മുതൽ വീണ്ടും traverse ചെയ്യണം. List വീണ്ടും access ചെയ്യാൻ head pointer സൂക്ഷിച്ചിരിക്കണം.

Q68 Define Circular List.

Answer: A circular linked list is a list in which the link of the last node points back to the first node instead of NULL.

മലയാളം: അവസാന node-ന്റെ link NULL ആകാതെ ആദ്യ node-ലേക്ക് തിരികെ point ചെയ്യുന്ന Linked List ആണ് Circular Linked List.

Q69 What are the disadvantages of a Circular List?

Answer: A singly circular list cannot be traversed backward. Care is needed when traversing because there is no NULL at the end; the algorithm must detect when it has returned to the starting node.

മലയാളം: Singly Circular List-ൽ പിന്നിലേക്ക് traverse ചെയ്യാൻ കഴിയില്ല. അവസാനം NULL ഇല്ലാത്തതിനാൽ ആദ്യ node-ലേക്ക് തിരികെ എത്തിയെന്ന് തിരിച്ചറിഞ്ഞ് traversal നിർത്തണം.

Q78 Which pointer type is used for a heterogeneous Linked List in C?

Answer: A void pointer (void *) can store the address of an object of any data type. The program must keep track of the actual type before converting and using the pointer.

മലയാളം: വ്യത്യസ്ത data types ഉള്ള objects-ന്റെ address സൂക്ഷിക്കാൻ C-യിൽ void pointer (void *) ഉപയോഗിക്കാം. ഉപയോഗിക്കുന്നതിന് മുമ്പ് യഥാർത്ഥ type അറിയുകയും ശരിയായി convert ചെയ്യുകയും വേണം.

Q93 Is a Linked List linear or non-linear?

Answer: In the standard classification, a linked list is a linear data structure because nodes form a sequence. The uploaded bank also contains a different storage-based wording; follow the definition taught by your instructor if they expect that wording.

മലയാളം: സാധാരണ classification പ്രകാരം Linked List linear data structure ആണ്; nodes ഒരു sequence രൂപത്തിലാണ്. നിങ്ങളുടെ question bank-ൽ storage അടിസ്ഥാനമാക്കി വേറൊരു wording ഉണ്ട്. Teacher പഠിപ്പിച്ച ഉത്തരവും ശ്രദ്ധിക്കുക.

Note: *The source question bank gives a conflicting classification in Q9 and Q93; this note makes the inconsistency explicit.*

DOUBLY LINKED LIST | Previous and Next Pointers

Q11 What is a Doubly Linked List? Give examples.

Answer: A doubly linked list is a linked list in which each node has links to both the next node and the previous node. It supports forward and backward traversal. Examples include browser back/forward navigation and playlists with next/previous controls.

മലയാളം: ഓരോ node-ലും അടുത്ത node-ന്റെയും മുൻ node-ന്റെയും address സൂക്ഷിക്കുന്ന Linked List ആണ് Doubly Linked List. മുന്നോട്ടും പിന്നോട്ടും traverse ചെയ്യാം. Browser back/forward, music playlist എന്നിവ ഉദാഹരണങ്ങളാണ്.

Q70 Define a Doubly Linked List.

Answer: A doubly linked list node has three parts: an information field, a pointer to the previous node and a pointer to the next node.

മലയാളം: Doubly Linked List-ലെ ഓരോ node-ലും മൂന്ന് ഭാഗങ്ങളുണ്ട്: data/info, previous node-ലേക്കുള്ള pointer, next node-ലേക്കുള്ള pointer.

BST DICTIONARY | Binary Search Tree

Q35 List the types of Trees.

Answer: Tree types include general trees, binary trees, Binary Search Trees (BSTs), AVL trees, red-black trees and N-ary trees. A binary tree has at most two children per node.

മലയാളം: General Tree, Binary Tree, BST, AVL Tree, Red-Black Tree, N-ary Tree എന്നിവ tree-കളുടെ ചില തരങ്ങളാണ്. Binary Tree-യിൽ ഒരു node-ന് പരമാവധി രണ്ട് children മാത്രമേ ഉണ്ടാകൂ.

Note: *The uploaded bank says a binary-tree parent has 'at least two' children; the correct definition is 'at most two'.*

Q36 What is a Binary Tree?

Answer: A binary tree is a tree in which each node has at most two children, called the left child and the right child. The top node is called the root.

മലയാളം: ഓരോ node-നും പരമാവധി രണ്ട് children (left, right) ഉള്ള tree ആണ് Binary Tree. ഏറ്റവും മുകളിലുള്ള node-നെ Root എന്ന് വിളിക്കുന്നു.

Q38 What are the advantages of Binary Search over Linear Search?

Answer: For a sorted array or a structure with efficient middle access, Binary Search takes O(log n) time by eliminating half the search range at each comparison. Linear Search takes O(n) time. Binary Search requires sorted data and efficient access to the middle.

മലയാളം: Sorted array-ൽ Binary Search ഓരോ comparison-ലും search range പകുതിയാക്കുന്നതിനാൽ O(log n) സമയം എടുക്കും. Linear Search O(n) ആണ്. Binary Search-ന് sorted data ആവശ്യമാണ്.

Q39 What is an AVL Tree?

Answer: An AVL tree is a self-balancing Binary Search Tree. For each node, the heights of the left and right subtrees differ by at most one, so the tree does not become highly skewed.

മലയാളം: AVL Tree ഒരു self-balancing BST ആണ്. ഓരോ node-ന്റെയും left, right subtree heights തമ്മിലുള്ള വ്യത്യാസം പരമാവധി 1 ആയിരിക്കും. അതിനാൽ tree വളരെയധികം skewed ആകില്ല.

Q46 How do you find the height of a node in a Tree?

Answer: The height of a node is the number of edges on the longest downward path from that node to a leaf. A leaf node has height 0 under this definition.

മലയാളം: ഒരു node മുതൽ അതിന്റെ താഴെയുള്ള leaf വരെ ഉള്ള ഏറ്റവും നീളമുള്ള പാതയിലെ edges എണ്ണമാണ് ആ node-ന്റെ height. ഈ definition പ്രകാരം leaf node-ന്റെ height 0 ആണ്.

Q84 List some applications of Tree data structures.

Answer: Trees are used to represent arithmetic expressions, construct symbol tables and support syntax analysis in compilers. They are also used in hierarchical data representation.

മലയാളം: Arithmetic expression പ്രതിനിധീകരിക്കാൻ, symbol table നിർമ്മിക്കാൻ, compiler-ൽ syntax analysis നടത്താൻ, hierarchical data സൂക്ഷിക്കാൻ Tree ഉപയോഗിക്കുന്നു.

P13 What is the main rule of a Binary Search Tree?

Answer: For each node, values in the left subtree are smaller than the node's key and values in the right subtree are greater than the node's key (assuming no duplicate-key rule).

മലയാളം: BST-ൽ ഒരു node-നെക്കാൾ ചെറിയ values left subtree-ലും വലിയ values right subtree-ലും വരും.

Note: *Additional practical question based on your BST dictionary program.*

P14 How does inorder traversal display a BST dictionary alphabetically?

Answer: Inorder traversal visits Left subtree, Root, then Right subtree. Since BST keys are ordered, this traversal displays the words in ascending order.

മലയാളം: Inorder traversal Left, Root, Right എന്ന ക്രമത്തിലാണ്. BST-യുടെ ordering rule കാരണം dictionary words ascending alphabetical order-ൽ ലഭിക്കും.

Note: *Additional practical question based on your BST dictionary program.*

PART 2 | OTHER GENERAL VIVA QUESTIONS

These questions are the remaining items from the uploaded viva bank that are not placed under the eight program-focused groups above.

GENERAL DATA STRUCTURE FUNDAMENTALS

Q1 What is a Data Structure?

Answer: A data structure is a way of organizing and storing data so that it can be accessed, updated and processed efficiently.

മലയാളം: Data കാര്യക്ഷമമായി സൂക്ഷിക്കാനും access ചെയ്യാനും മാറ്റാനും process ചെയ്യാനും ഉപയോഗിക്കുന്ന ക്രമീകരണമാണ് Data Structure.

Q2 Describe the types of Data Structures.

Answer: Common forms include arrays, lists, records/structures, trees and tables. A broad classification is linear and non-linear data structures.

മലയാളം: Array, List, Record/Structure, Tree, Table എന്നിവ data organization-ന്റെ രൂപങ്ങളാണ്. പൊതുവായി Linear, Non-linear എന്നിങ്ങനെ വിഭജിക്കാം.

Q3 What is a Linear Data Structure? Give examples.

Answer: A linear data structure arranges elements in a sequence. Except for the first and last elements, each element has a preceding and following element. Examples include arrays, stacks, queues and linked lists.

മലയാളം: Elements ഒരു ക്രമത്തിൽ നിരത്തിയിരിക്കുന്ന structure ആണ് Linear Data Structure. Array, Stack, Queue, Linked List എന്നിവ ഉദാഹരണങ്ങളാണ്.

Q4 What are some applications of Data Structures?

Answer: Data structures are used in operating systems, compiler design, database management, graphics, artificial intelligence, numerical analysis, statistics and simulation.

മലയാളം: Operating System, Compiler Design, Database, Graphics, AI, Numerical Analysis, Statistics, Simulation എന്നിവയിൽ Data Structures ഉപയോഗിക്കുന്നു.

Q5 What is the difference between File Structure and Storage Structure?

Answer: Storage structure refers to the organization of data in main memory. File structure refers to how data is organized in auxiliary or secondary storage, such as files on a disk.

മലയാളം: Storage Structure എന്നത് computer main memory-യിലെ data organization ആണ്. File Structure എന്നത് disk പോലുള്ള secondary storage-ലെ data organization ആണ്.

Q12 How do you reference all elements in a one-dimensional array?

Answer: Use an indexed loop. Start the index at 0 and continue until n - 1, accessing each element as array[index].

മലയാളം: Index ഉപയോഗിച്ചുള്ള loop ഉപയോഗിക്കാം. 0 മുതൽ n-1 വരെ index മാറ്റി ഓരോ array element-ഉം access ചെയ്യാം.

Q64 What are the disadvantages of sequential storage?

Answer: Sequential storage reserves a fixed amount of memory. If fewer elements are stored, some allocated space may be unused; if more elements are needed than the capacity allows, overflow can occur.

മലയാളം: Sequential Storage-ൽ memory-യുടെ ഒരു fixed amount മുൻകൂട്ടി reserve ചെയ്യുന്നു. കുറച്ച് elements മാത്രം ഉപയോഗിച്ചാൽ space പാഴാകാം; capacity-നേക്കാൾ കൂടുതൽ elements വേണമെങ്കിൽ overflow സംഭവിക്കാം.

Q24 What are the advantages of Heap over Stack?

Answer: For dynamic memory, the heap supports allocation and deallocation of blocks at run time and is suitable for objects whose lifetime is not limited to one function call. Stack memory is automatically managed for local variables and function calls. They serve different purposes.

മലയാളം: Heap memory run time-ൽ ആവശ്യത്തിന് allocate/free ചെയ്യാം; function call കഴിഞ്ഞാലും നിലനിർത്തേണ്ട objects-ന് അനുയോജ്യമാണ്. Stack local variables-നും function calls-നും ഉപയോഗിക്കുന്നു. രണ്ടിനും വ്യത്യസ്ത ആവശ്യങ്ങളാണ്.

Note: *The source uses 'Heap' and 'Stack' as memory areas here, not as the Heap data structure.*

Q47 What is Data Structure?

Answer: A data structure is a logical and mathematical way of organizing data. A broad classification divides data structures into linear and non-linear types.

മലയാളം: Data ക്രമമായി organize ചെയ്യുന്ന logical/mathematical രീതിയാണ് Data Structure. Linear, Non-linear എന്നിങ്ങനെ പ്രധാനമായി തരംതിരിക്കുന്നു.

Q48 What are the goals of Data Structures?

Answer: A data structure should represent relationships among real-world data clearly while remaining simple enough for efficient processing.

മലയാളം: യഥാർത്ഥ ലോകത്തിലെ data ബന്ധങ്ങൾ ശരിയായി പ്രതിനിധീകരിക്കണം; അതേ സമയം processing കാര്യക്ഷമമാക്കാൻ structure ലളിതവുമായിരിക്കണം.

Q49 What does Abstract Data Type mean?

Answer: An Abstract Data Type (ADT) specifies a set of values and the operations allowed on them, without describing how they are implemented. It defines the logical properties of a data type.

മലയാളം: ഒരു data type-ൽ ഏതെല്ലാം values ഉം operations ഉം ഉണ്ടെന്ന് implementation വിശദാംശങ്ങൾ പറയാതെ നിർവചിക്കുന്നതാണ് Abstract Data Type (ADT).

Q50 What is the difference between a Stack and an Array?

Answer: An array is an indexed collection of elements, usually of the same type and with a declared capacity. A stack is an abstract data type that restricts insertion and deletion to the TOP and follows LIFO; it can be implemented using an array or a linked list.

മലയാളം: Array-ൽ index ഉപയോഗിച്ച് elements access ചെയ്യാം. Stack-ൽ insertion/deletion TOP-ൽ മാത്രം; LIFO rule ആണ്. Stack array ഉപയോഗിച്ചോ linked list ഉപയോഗിച്ചോ implement ചെയ്യാം.

Q51 What is a Recursive Definition?

Answer: A recursive definition describes an object or process in terms of smaller or simpler instances of itself, together with a base case.

മലയാളം: ഒരു object-നെയോ process-നെയോ അതിന്റെ തന്നെ കൂടുതൽ ലളിതമായ രൂപങ്ങൾ ഉപയോഗിച്ച് നിർവചിക്കുന്നതാണ് Recursive Definition. നിർത്താനുള്ള base case ആവശ്യമാണ്.

Q53 What actions are performed when a function is called?

Answer: Arguments are passed, local variables and the function's execution context are set up, and control transfers to the called function.

മലയാളം: Function വിളിക്കുമ്പോൾ arguments pass ചെയ്യുന്നു, local variables/execution context ഒരുക്കുന്നു, തുടർന്ന് control ആ function-ലേക്ക് മാറുന്നു.

Q54 What actions are performed when a function returns?

Answer: The function's local execution context is released, the return value (if any) is provided, and control returns to the saved return address in the calling function.

മലയാളം: Function return ചെയ്യുമ്പോൾ അതിന്റെ local execution context ഒഴിവാക്കുന്നു; return value ഉണ്ടെങ്കിൽ നൽകുന്നു; തുടർന്ന് calling function-ലേക്ക് control മടങ്ങുന്നു.

Q76 List the areas in which Data Structures are applied extensively.

Answer: Applications include compiler design, operating systems, database management systems, statistical packages, numerical analysis, graphics, artificial intelligence and simulation.

മലയാളം: Compiler Design, Operating System, DBMS, Statistical packages, Numerical Analysis, Graphics, AI, Simulation എന്നിവയിൽ Data Structures വ്യാപകമായി ഉപയോഗിക്കുന്നു.

OTHER SORTING AND SEARCHING QUESTIONS

Q29 How does Selection Sort work?

Answer: Selection Sort repeatedly finds the smallest element in the unsorted part and swaps it into the next position at the beginning. Its time complexity is O(n^2) and its extra space complexity is O(1).

മലയാളം: Selection Sort-ൽ sort ചെയ്യാത്ത ഭാഗത്തിലെ ഏറ്റവും ചെറിയ element കണ്ടെത്തി അടുത്ത ശരിയായ സ്ഥാനത്തേക്ക് swap ചെയ്യുന്നു. Time O(n^2), extra space O(1) ആണ്.

Q52 What is Sequential Search?

Answer: Sequential (Linear) Search checks elements one by one until the required item is found or the list ends. It can be used on an array or linked list.

മലയാളം: Sequential/Linear Search-ൽ ഓരോ element-ഉം ഒന്നൊന്നായി പരിശോധിക്കുന്നു. Item കിട്ടുന്നതുവരെ അല്ലെങ്കിൽ list തീരുന്നതുവരെ തുടരുന്നു.

Q71 Is it necessary to sort a file before searching for an item?

Answer: Not always. If only a few searches are needed, sorting first may cost more than it saves. If many searches will be performed, sorting once can make later searches more efficient.

മലയാളം: Search കുറച്ച് തവണ മാത്രം ആണെങ്കിൽ ആദ്യം sort ചെയ്യുന്നത് അധിക ജോലിയാകാം. പലതവണ search ചെയ്യേണ്ടിവന്നാൽ ഒരിക്കൽ sort ചെയ്യുന്നത് പിന്നീട് കാര്യക്ഷമമാകും.

Q73 What is the efficiency of Sequential Search?

Answer: Best case: 1 comparison. Worst case: n comparisons. Average successful search in an equally likely position: (n + 1) / 2 comparisons. Its time complexity is O(n).

മലയാളം: ആദ്യ സ്ഥാനത്ത് കിട്ടിയാൽ 1 comparison; അവസാനത്തോ ഇല്ലാതെയോ ആണെങ്കിൽ n comparisons വരെ. Average successful search ഏകദേശം (n+1)/2. Time O(n) ആണ്.

OTHER TREES, GRAPHS AND STRUCTURES

Q33 Define a Graph data structure.

Answer: A graph is a non-linear data structure consisting of vertices (nodes) connected by edges. Edges may be directed or undirected.

മലയാളം: Vertices/nodes-ഉം അവയെ ബന്ധിപ്പിക്കുന്ന edges-ഉം ചേർന്ന non-linear data structure ആണ് Graph. Edges directed അല്ലെങ്കിൽ undirected ആകാം.

Q34 What are the applications of Graph data structures?

Answer: Graphs model transport networks, power or water networks, social networks and neural networks. Vertices represent entities and edges represent relationships or connections.

മലയാളം: Transport routes, electricity/water networks, social networks, neural networks എന്നിവ Graph ഉപയോഗിച്ച് പ്രതിനിധീകരിക്കാം. Vertices objects-ഉം edges ബന്ധങ്ങളും സൂചിപ്പിക്കുന്നു.

Q37 What are the differences between B Tree and B+ Tree?

Answer: In a B-tree, keys and associated records/data may be stored in internal as well as leaf nodes. In a B+ tree, records are stored at the leaf level and internal nodes guide the search; leaves are commonly linked for range access.

മലയാളം: B-Tree-ൽ keys/data internal nodes-ലും leaf nodes-ലും ഉണ്ടാകാം. B+ Tree-ൽ records സാധാരണ leaf nodes-ലാണ്; internal nodes search guide ചെയ്യുന്നു. Leaf nodes range search-നായി ബന്ധിപ്പിക്കാറുണ്ട്.

Q43 List some applications of Multilinked Structures.

Answer: Examples include sparse matrix representation and index generation, as listed in the question bank.

മലയാളം: Sparse Matrix representation, Index Generation എന്നിവ Multilinked Structures-ന്റെ ഉദാഹരണങ്ങളാണ്.

Q44 Explain a Jagged Array.

Answer: A jagged array is an array whose elements are themselves arrays, and those inner arrays may have different lengths or sizes.

മലയാളം: ഓരോ element-ും മറ്റൊരു array ആയിരിക്കാം; ആ inner arrays-ന്റെ size ഒരുപോലെ ആവണമെന്നില്ല. അത്തരത്തിലുള്ള array ആണ് Jagged Array.

Q45 Explain a Max Heap data structure.

Answer: A Max Heap is a complete binary tree in which every parent node is greater than or equal to its children. Therefore, the maximum value is at the root.

മലയാളം: ഓരോ parent node-ന്റെയും value children-നെക്കാൾ വലുതോ തുല്യമോ ആയ complete binary tree ആണ് Max Heap. ഏറ്റവും വലിയ value root-ൽ ആയിരിക്കും.

Q77 What data structures are used for Network and Hierarchical data models?

Answer: A graph is used to represent a network data model. A tree is used to represent a hierarchical data model.

മലയാളം: Network data model പ്രതിനിധീകരിക്കാൻ Graph ഉപയോഗിക്കുന്നു. Hierarchical data model പ്രതിനിധീകരിക്കാൻ Tree ഉപയോഗിക്കുന്നു.

Q85 List applications of Multilinked Structures.

Answer: The question bank lists sparse matrices and index generation as applications.

മലയാളം: Sparse Matrix-ുകളും Index Generation-ും Multilinked Structures-ന്റെ ഉപയോഗങ്ങളാണ്.

Q86 Which data structure is suitable for tree construction?

Answer: The uploaded question bank gives Linked List as the expected option. In pointer-based tree implementations, each node is dynamically allocated and connected with pointers, which is a linked representation.

മലയാളം: നിങ്ങളുടെ question bank-ൽ പ്രതീക്ഷിക്കുന്ന ഉത്തരം Linked List എന്നാണ്. Pointer-based tree implementation-ൽ ഓരോ node-ും memory allocate ചെയ്ത് pointers ഉപയോഗിച്ച് ബന്ധിപ്പിക്കുന്നു.

Q87 What type of algorithm is used to solve the 8 Queens problem?

Answer: The 8 Queens problem is commonly solved using backtracking. A queen is placed, constraints are checked, and the algorithm backtracks when a placement leads to a conflict.

മലയാളം: 8 Queens problem സാധാരണ Backtracking ഉപയോഗിച്ചാണ് പരിഹരിക്കുന്നത്. Queen വയ്ക്കുന്നു, conflict ഉണ്ടോ നോക്കുന്നു; പ്രശ്നമുണ്ടെങ്കിൽ മുൻപത്തെ നിലയിലേക്ക് മടങ്ങി മറ്റൊരു സ്ഥാനമെടുക്കുന്നു.

Q88 In an AVL Tree, when is balancing required?

Answer: Balancing is required when the balance factor of a node becomes less than -1 or greater than +1. A balanced AVL node has a balance factor of -1, 0 or +1.

മലയാളം: ഒരു node-ന്റെ balance factor -1-ൽ താഴെയോ +1-ൽ കൂടുതലോ ആയാൽ balancing വേണം. AVL-ൽ balance factor -1, 0, +1 എന്നിവയിൽ ഒന്നായിരിക്കണം.

Note: *The question bank's wording is imprecise. The standard AVL condition is |balance factor| > 1.*

Q89 In RDBMS, what efficient data structure is used for internal storage representation?

Answer: The question bank gives B+ tree. B+ trees support efficient searching and range queries; data records are stored at the leaf level in common database implementations.

മലയാളം: RDBMS-ൽ B+ Tree ഉപയോഗിക്കുന്നതായി question bank പറയുന്നു. Search-ഉം range query-കളും കാര്യക്ഷമമാക്കാൻ B+ Tree സഹായിക്കുന്നു; records leaf level-ലാണ് സൂക്ഷിക്കുന്നത്.

Q90 Which tree structure is efficient considering space and time complexities: incomplete, complete or full binary tree?

Answer: The question bank gives Complete Binary Tree as the expected answer. A complete binary tree fills levels from left to right, with only the final level allowed to be incomplete.

മലയാളം: നിങ്ങളുടെ question bank-ൽ expected answer Complete Binary Tree ആണ്. അവസാന level ഒഴികെ എല്ലാ levels-ഉം നിറഞ്ഞിരിക്കും; അവസാന level ഇടത്തുനിന്ന് വലത്തോട്ട് നിറയും.

Q91 What is a Spanning Tree?

Answer: A spanning tree of a connected graph includes all vertices and has no cycles. A minimum spanning tree is a spanning tree with the smallest possible total edge weight.

മലയാളം: Connected graph-യിലെ എല്ലാ vertices-ഉം ഉൾപ്പെടുത്തി cycle ഇല്ലാത്ത tree ആണ് Spanning Tree. ആകെ edge weight ഏറ്റവും കുറവുള്ളത് Minimum Spanning Tree ആണ്.

Q92 Does a Minimum Spanning Tree give the shortest distance between any two specified nodes?

Answer: No. A minimum spanning tree minimizes the total weight of all selected edges. It does not necessarily provide the shortest path between every pair of vertices.

മലയാളം: ഇല്ല. Minimum Spanning Tree എല്ലാ തിരഞ്ഞെടുത്ത edges-ന്റെയും മൊത്തം weight കുറയ്ക്കുന്നു. എന്നാൽ ഓരോ രണ്ട് nodes-നും ഇടയിലെ shortest path ഉറപ്പാക്കുന്നില്ല.

OTHER RECURSION AND FUNCTION QUESTIONS

Q74 Are implicit arguments passed to a function when it is called?

Answer: A function call uses execution information such as a return address and calling context. These details are managed by the language implementation and calling convention; they are not ordinary arguments written in the function declaration.

മലയാളം: Function call-ൽ return address പോലുള്ള execution വിവരങ്ങൾ ആവശ്യമാണ്. ഇവ compiler/calling convention manage ചെയ്യുന്നു; function declaration-ൽ എഴുതുന്ന സാധാരണ arguments അല്ല.

Q80 What data structure is used to perform Recursion?

Answer: Recursion uses the call stack. Each function call stores information such as local variables and the return address so execution can resume after the call finishes.

മലയാളം: Recursion പ്രവർത്തിക്കുമ്പോൾ Call Stack ഉപയോഗിക്കുന്നു. ഓരോ function call-ന്റെയും local variables, return address തുടങ്ങിയ വിവരങ്ങൾ stack-ൽ സൂക്ഷിക്കുന്നു.

Last-minute revision checklist

Insertion Sort: Key, compare, shift larger elements right, insert key. Best O(n); average/worst O(n^2).

Quick Sort: Pivot, partition, recursive calls. Average O(n log n); worst O(n^2).

Merge Sort: Divide, sort each half, merge. O(n log n) time; typically O(n) extra array space.

Infix to Postfix: Stack, operator precedence, associativity, parentheses, PUSH and POP.

Circular Queue: FRONT, REAR, count, overflow/underflow, modulo wrap-around formula.

Singly Linked List: Node has data and next link; head points to first node; last link is NULL.

Doubly Linked List: Node has data, previous pointer and next pointer; traversal works both ways.

BST Dictionary: Smaller keys left, greater keys right; search by comparison; inorder gives sorted order.

Study tip: For every program, practise explaining the aim, main data structure, important variables, menu operations, one dry run and time complexity.